

Operating System Installation

René Serral-Gracià¹

¹Universitat Politècnica de Catalunya (UPC)

February 16, 2026

Lectures

- 1 System administration introduction
- 2 **Operating System installation**
- 3 User management
- 4 Application management
- 5 System monitoring
- 6 Filesystem Maintenance
- 7 Network services
- 8 Security and Protection
- 9 Introduction to Public Cloud

Outline

- 1 Introduction
- 2 Equipment Life-cycle
- 3 System installation
- 4 Disk Partitioning and filesystems
- 5 System Init/Shutdown

Outline

- 1 Introduction
 - Goals
- 2 Equipment Life-cycle
- 3 System installation
- 4 Disk Partitioning and filesystems
- 5 System Init/Shutdown

Goals

Abilities

- Installation scheduling
 - Disk Partitioning
 - File System creation
 - Swap area dimensioning
- Basic configuration
 - System Startup and Shutdown

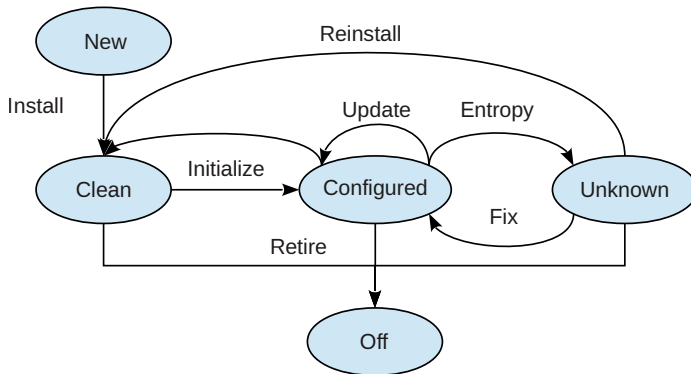
Configuration Commands and files

- `fdisk`, `mkfs`, `mkswap`, `mount`, `swapon`
- `shutdown`, `halt`, `reboot`, `poweroff`
- `systemd`, `/etc/systemd`, `/etc/fstab`

Outline

- 1 Introduction
- 2 Equipment Life-cycle**
- 3 System installation
- 4 Disk Partitioning and filesystems
- 5 System Init/Shutdown

Equipment Life-cycle¹



- Sysadmin goals:

- Understand the existence of the states and their transitions
- Maximize the amount of time in the *“Configured”* state

¹ Rémy Evard. “An analysis of UNIX system configuration”. 11th Systems Administration Conference (LISA 97)

Equipment Life-cycle

States

- **New:** new equipment
- **Clean:** equipment with the installed OS but without any maintenance task
- **Configured:** configured equipment according to the environment requirements
- **Unknown:** unconfigured or outdated equipment
- **Off:** discarded equipment due to its age or hardware failure

Equipment Life-cycle

Transitions

- **Install:** OS installation
- **Initialize:** Initial set of required changes to have the equipment configured in the work environment
- **Update:** Insert new functionalities, apply patches and security updates
- **Entropy:** Gradual degradation process leaving the equipment in unknown state
- **Fix:** take the necessary actions to set the equipment back to configured state
- **Reinstall:** massive update of the OS. Usually forced by an attack, goal shift in the equipment, or configuration errors
- **Retire:** final retirement of the equipment

Outline

- 1 Introduction
- 2 Equipment Life-cycle
- 3 System installation**
 - Previous tasks
 - Installation
- 4 Disk Partitioning and filesystems
- 5 System Init/Shutdown

System installation

- 1 Goals
- 2 Dimensioning
- 3 HW Acquisition
- 4 Disk preparation
- 5 Protected network setup
- 6 Install / OS & Software update
- 7 Service configuration / adaptation
- 8 Security policy enforcement
- 9 Final location network setup
- 10 Label / Document the followed steps
- 11 Monitor. . . goto 5

Previous tasks

1 Goals

Which is the purpose of the new equipment?

- Desktop
 - Document editing?
 - Compiling?
- Server
 - E-mail? Web? Proxy? DNS? Files?
 - Primary? Secondary?
- Amount of expected users
- Security requirements

Previous tasks

2 Dimensioning

- CPU
- Memory
- Disk
- Redundancy

3 Buy HW

- OS Compatibility (drivers!)
- List of features
 - IRQs, DMA, and/or ports...

Installation

- ④ Disk preparation
 - Partitioning
 - Swap area preparation
 - Format and prepare the filesystems
- ⑤ Connect the equipment into a secure network
 - So during the installation the machine is protected
- ⑥ Install / Update OS & Software
 - Choose OS / Distribution
 - Select the package update list

Installation

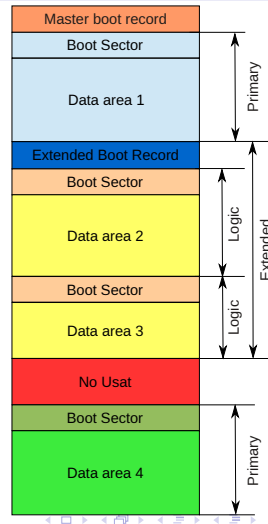
- 7 Service configuration
 - Adapt them to the work environment
- 8 Implement security policies
 - Offer only the necessary services
- 9 Connect to the network
 - To the final location
- 10 Label / Document the followed steps
 - In case it is necessary to repeat them, to apply them on other machines, ...
- 11 Monitoring... goto 6

Outline

- 1 Introduction
- 2 Equipment Life-cycle
- 3 System installation
- 4 Disk Partitioning and filesystems**
 - Filesystem preparation/format
 - Swap area
- 5 System Init/Shutdown

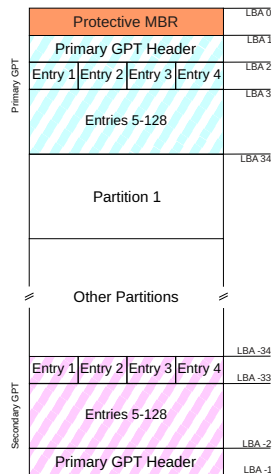
Master Boot Record (MBR) – Outdated

- Up to 4 “primary” partitions in the Master Boot Record
 - Or 3 primary and 1 extended
- Primary partition
 - May contain a filesystem
- Extended partition
 - Can only contain logical partitions
- Logical partitions
 - May contain a filesystem



Types of partitions – GUID Partition Table (GPT)

- Up to 128 partitions with the default size of GPT
- There is no distinction of primary and extended partitions anymore, now it is identified by UUID
 - The partition type is determined by the Operating System, which assigns its own IDs



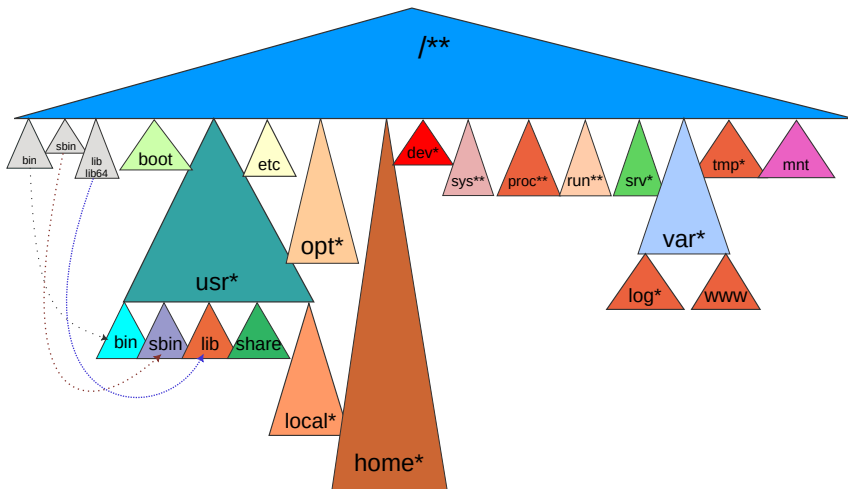
Partitions: concept and justification

Divide one disk into several independent disks

- Each partition is completely isolated from the others
 - Error isolation
 - More security
- Backup management different for each partition
 - Faster
 - More convenient
 - *Read-only* or not much changed partitions
- Information reuse among OS

Problem: hard disk fragmentation

Filesystem structure in UNIX



* Can be mounted filesystems

Filesystem structure in UNIX

- **/bin and /sbin**
 - Executables needed during boot time
 - ip, mount, ls, cat, ...
- **/usr/bin and /usr/sbin**
 - Operating system applications
 - man, apropos, ...
 - adduser, deluser, ...
- **/usr/local/bin and /usr/local/sbin (or /opt)**
 - Specific applications
- **\$HOME/bin**
 - End-user applications

Filesystem structure in UNIX

`/var`

- Dynamic content
 - Accounting
 - Information about end-user activity
 - Spool
 - Mail
 - Cron/at
 - cups
 - Run
 - Pid's of running daemons
 - Log
 - System logs

Filesystem preparation/format

- `mkfs -t tipus [opcions] dispositiu`
 - **type:** ext4, vfat, btrfs, zfs,...
 - **options** (filesystem dependent)
 - block size
 - number of inode
 - number of blocks (usually autodetected)
 - ...
- `tune2fs [-l] [-j] ...`
 - Filesystem ext[234] parameter configuration
 - Filesystem check interval
 - Journal creation
 - ...

Exercise – En grup

- If we put all the directories labelled with * and ** in their own partition. Determine a correct size for each partition
- Why the rest of the directoris cannot be on a partition by themselves

Exercise – En grup

- If we put all the directories labelled with * and ** in their own partition. Determine a correct size for each partition
 - Such size normally depends of the particular needs for that installation. Usually a regular Linux installation needs around 15GB
- Why the rest of the directoris cannot be on a partition by themselves
 - The content is necessary during the boot process. Potentially before mounting the filesystems

Mount

- `mount` *[options] device directory*
 - `-t <filesystem type>`
 - Indicate the type of the filesystem
 - `-a`
 - mount all the filesystems in `/etc/fstab`
 - `-o <FS options>`
 - `ro` = read-only
 - `remount`
 - `noexec`, `nodev`, `nosuid`
 - `user`

/etc/fstab

- Indicates how to mount the filesystems

Device	M. point	FS	Options	D	F
/dev/sda1	/boot	vfat	defaults	0	2
/dev/sda2	/	btrfs	defaults	0	1
/dev/sda3	/var	ext4	defaults	0	2
/dev/sda4	/home	ext4	defaults	0	2
/dev/sda5	swap	swap	defaults	0	0

- Nowadays `/etc/fstab` is partially deprecated in favor of `systemd.mount` a utility based on `systemd` to handle filesystem mounts in modern systems

Exercise – In group

- We have a server with 100 users, with a disk quota of 5GB per user. The system has a 1TB harddisk. Indicate how can you partition it and the size of each partition.

Exercise – In group

- We have a server with 100 users, with a disk quota of 5GB per user. The system has a 1TB harddisk. Indicate how can you partition it and the size of each partition.
 - The users need a total of $\sim 500GB$. $\sim 30GB$ for the base system², then lacking more information we leave a total of $\sim 40GB$ for applications.
 - We will have 3 different partitions, the root partition `/dev/sda1` with 30GB, the user's partition `/dev/sda2` using 600Gb, 40Gb for applications `/dev/sda5`, and finally 32GB for the swap partition `/dev/sda6`. We leave the rest of the disk unpartitioned
 - For safety we leave a threshold of 10 – 20% in terms of space for each partition

²Assuming a Linux Debian installation

Exercise – In group

- List the required commands in order to be able to mount the filesystems indicated in the previous exercise, knowing that the application partition must be read-only.

Exercise – In group

- List the required commands in order to be able to mount the filesystems indicated in the previous exercise, knowing that the application partition must be read-only.
 - `/dev/sda1` → it must be mounted from `/etc/fstab`
 - `/dev/sda2` → `mount /dev/sda2 /home`
 - `/dev/sda5` → `mount -o ro /dev/sda5 /usr`

Exercise – In group

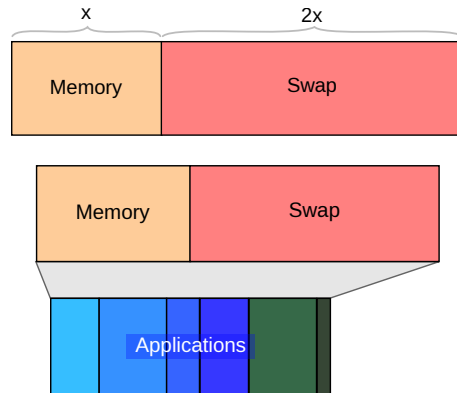
- List the required commands in order to be able to mount the filesystems indicated in the previous exercise, knowing that the application partition must be read-only.
 - `/dev/sda1` → it must be mounted from `/etc/fstab`
 - `/dev/sda2` → `mount /dev/sda2 /home`
 - `/dev/sda5` → `mount -o ro /dev/sda5 /usr`
- Can you devise any situation where more partitions could be necessary?

Exercise – In group

- List the required commands in order to be able to mount the filesystems indicated in the previous exercise, knowing that the application partition must be read-only.
 - `/dev/sda1` → it must be mounted from `/etc/fstab`
 - `/dev/sda2` → `mount /dev/sda2 /home`
 - `/dev/sda5` → `mount -o ro /dev/sda5 /usr`
- Can you devise any situation where more partitions could be necessary?
 - If the server had some specific requirements, for example a very large web page, we could be interested in having `/var/www` in a different partition

Swap area

- Rule of thumb
 - $\text{Swap} = 2 * \text{physical memory}$
- Realment
 - Foresee memory requirements and choose it accordingly



Swap area implementation

- As a disk partition
 - Better if divided into multiple devices
- Special file
 - Pre-created and completely reserved... it cannot have any “holes”
 - Holes??? in a file???
 - ```
dd if=/dev/zero of=swapfile bs=1024 count=65536
```
  - **Be careful!**
    - File protections
    - The is sensible information from the swapped out processes

# Swap area Creation/Preparation

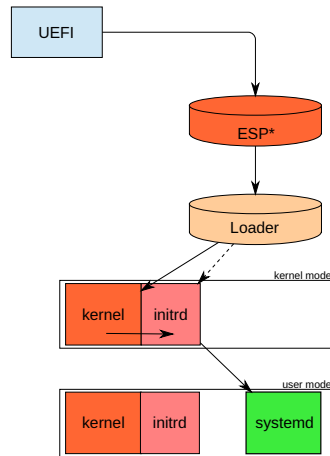
- `mkswap` *device | file*
  - Creates a swap area — is equivalent to swap area “format”
- `swapon` [*options*] [*device | file*]
  - `-p` *priority*
    - The swap with more priority is used before
    - Round-Robin if equal priority
  - `-a`
    - Activates all the swaps defined in `/etc/fstab`
- `swapoff` [*options*] [*device | file*]
  - Disables a given swap area
  - `-a`
    - Disables all the ones defined into `/etc/fstab`

# Outline

- 1 Introduction
- 2 Equipment Life-cycle
- 3 System installation
- 4 Disk Partitioning and filesystems
- 5 System Init/Shutdown**
  - System initialization
  - System shutdown

# System initialization

- ROM
  - Hardware initialization
    - CPUs,...
- kernel
  - Hardware detection
  - Kernel mode configuration
- initrd
  - Device configuration
- init
  - User space configurations



\* EFI System Partition (ESP)

# systemd: Architecture and Lifecycle Management

## Core Concept

A system and service manager that acts as the `init` process (PID 1), replacing the legacy SysV `init` with a declarative, dependency-driven model

- **Parallelization via Sockets:** Uses *socket activation* to start services concurrently
- **Unit-Based Logic:** Everything is a "Unit" (`.service`, `.mount`, `.target`). Configuration is declarative (INI style) rather than imperative shell scripts
- **Cgroups for Tracking:** Leverages Linux Control Groups (cgroups) to track service processes
- **On-Demand Execution:** Services can be triggered by hardware events (`udev`), timer events, or bus activity (D-Bus)

# systemd

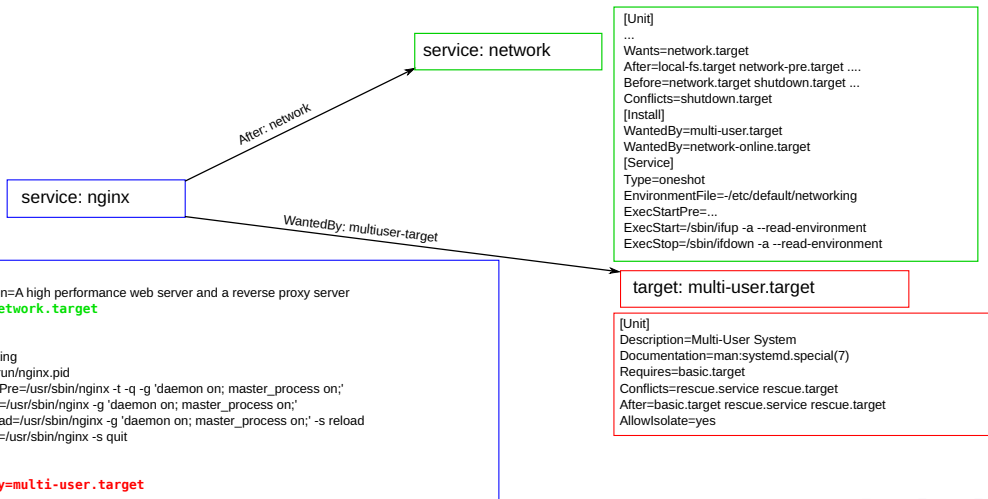
## The Dependency Graph

`systemd` treats the boot process as a **Directed Acyclic Graph (DAG)**

It resolves dependencies like `Wants=`, `Requires=`, and `After=` to optimize the execution path

- Only available for Linux
- Built-in hardware management through `udev`
- Boot based on dependencies and targets
- Is composed by a series of modules performing specific operations: `mount`, `udev`, `network`

# systemd - target based execution



# Systemd Units: .mount and .automount

## Underlying idea

Treats mount points as first-class units allowing the init system to manage filesystem dependencies like any other service

- **Declarative Naming:** Unit names must match the mount path
  - Path `/var/log` → Unit `var-log.mount`
- **Implicit Dependencies:** Systemd automatically generates dependencies. If a service requires `/data`, systemd ensures the mount is active before the daemon starts
- **Encapsulation of Options:** Replaces `/etc/fstab` entries with structured sections: `[Mount]`, `[What]`, `[Where]`, and `[Type]`
- **On-Demand Mounting:** `.automount` units use **autofs** logic to mount filesystems only upon first access, reducing boot-time latency for network shares (NFS/CIFS)

# systemd - Services management

- `systemctl`: **List all services and their state**
- `systemctl [start|stop|restart] <services>`:  
**Starts/Stops/Restarts the <service>**
- `systemctl reload <service>`: **Reread service configuration without stopping**
- `systemctl [enable|disable] <services>`: **Enables/Disables the service to be started on boot**
- `systemctl status <service>`: **Service status**
- `systemctl [mask|unmask] <service>`: **Masks/unmasks service execution**
- `systemctl daemon-reload`: **Self-reload of the service**

# systemd - tools

## systemd Utilities

systemctl journalctl notify analyze cglsg cgtop loginctl nspawn

## systemd Daemons

systemd  
journald networkd  
logind user session

## systemd Targets

bootmode basic multi-user graphical user-session  
shutdown reboot dbus telephony display service  
dlog logind user-session tizen service

## systemd Core

manager unit login namespace log  
systemd service timer mount target  
snapshot path socket swap  
multiseat inhibit session pam  
cgroup dbus

## systemd Libraries

dbus-1 libpam libcap libcryptsetup tcpwrapper libaudit libnotify

## Linux Kernel

cgroups autofs kdbus

# Journald: Structured System Logging

**Design Philosophy:** A centralized, high-performance logging daemon that treats log data as a **structured stream** rather than a flat text file.

- **Binary Format:** Logs are stored in a persistent, indexed binary format
- **Metadata Enrichment:** Automatically attaches trusted fields to every entry
- **Reliability:** Eliminates the "dropped logs" problem common with UDP-based syslog
- **Forwarding:** Can act as a buffer for traditional syslog daemons (`rsyslog`)

## The Power of journalctl

```
journalctl -u nginx.service -since "2026-01-01" -p err
```

*Querying indexed fields is significantly faster than `grep` over `/var/log/messages`*

# System shutdown

Performs the inverse operation than system booting

- Stop all services — Network + locals
- Stop all the processes
- Sync all buffer caches
- Umount all the filesystem
- Stop/reboot the system

## Commands

- `shutdown`: allows shutdown/reboot at a given time
- `reboot`, `halt`, `poweroff`, ...
- `systemctl reboot`, `systemctl poweroff`

# Personal work

- Privileges and protection
  - Owners and groups
  - Privileges (r, w, x)
  - umask
  - setuid, setgid
- User management related commands
  - chmod, chown, id, newgrp
  - useradd, userdel
  - chsh, passwd
  - groupadd, groupdel